Assignment 3: Load Balancing and Observability

Enterprise Software Development, Fall 2026

Individual assignment · Extend A2 · Duration: 1–2 weeks

Use Envoy to distribute HTTP requests and gRPC calls across replicas of your application. Use your A1/A2 observability stack to explain the results.

Part A: Replicas and routing

Run **three application replicas**, A, B, and C, using Docker Compose scaling. Each replica exposes the same HTTP endpoints and gRPC methods.

```
HTTP / gRPC clients ⟶ Envoy ⟶ Replica A: HTTP + gRPC

Replica B: running, no application traffic
Replica C: running, no application traffic
```

- Configure Envoy to forward both HTTP and gRPC to **A only**. Keep B and C running, but leave them out of Envoy's destinations for now.
- Give each replica an identifier in logs and metrics. Preserve request/trace correlation.
- Keep business rules and data consistent across replicas. For writes, preserve duplicate-prevention rules.

Use one HTTP operation and one unary gRPC operation from A2 as test traffic. Read-only operations are easiest.

Draw your architecture. For your data, rate-limit counters, and idempotency records, say whether each is shared by all replicas (for example, a database) or kept separately in each replica (for example, in memory). If any is kept separately, explain what goes wrong when requests are spread across replicas.

Part B: Observe the baseline

Build one Grafana dashboard that shows, for HTTP and gRPC, **per replica**:

- request rate and share of total traffic;
- active requests;
- p50/p95 response time (state the time window);
- errors by status.

Prometheus must scrape A, B, C, and Envoy **directly**, not through the balanced route.

Send traffic at a fixed rate for a fixed duration, without waiting for each response before sending the next. Record the rate, requests sent, successes, failures, and client p95.

Expected baseline: A handles all traffic; B and C are running but idle.

Ship Envoy's JSON access logs to Elasticsearch through Filebeat, including the selected replica and the trace ID. For each interface, show one search that returns the Envoy event together with the replica's event.

You will reuse this dashboard and workload in Parts C and D. Keep unique IDs out of metric labels.

Part C: Enable load balancing

Add B and C to Envoy's destinations for both HTTP and gRPC, using **equal-weight round robin**.

Repeat Part B's workload. **Expected outcome:** each replica receives about one-third of the traffic. Show this on the Part B dashboard, compare each replica's share with 33%, and explain any difference from the A-only baseline. These results are your round-robin baseline for Part D.

> **Note on gRPC.** A gRPC client opens one connection and sends every call through it. If Envoy proxies gRPC as plain TCP, it cannot see individual calls, so it chooses a replica once per connection: all calls go to one replica, and B and C stay idle.
>
> Configure Envoy's gRPC listener as HTTP/2 instead. Envoy can then see each call and route it separately, so calls spread across replicas. The client still uses one connection to Envoy; only the routing decision moves from per connection to per call.

Part D: Experiments and conclusions

Start each experiment from Part C's healthy round-robin setup, use the same workload, and restore normal behaviour afterwards. Report HTTP and gRPC separately. Disable client retries and circuit breakers during tests.

For each experiment, give a **prediction, observations, and explanation**, supported by charts and client results.

Experiment	Run	Explain
Weighted round robin	Set A:B:C weights to **2:1:1**.	Compare shares with **50% : 25% : 25%**; explain any change in latency and active requests.
Least request	Restore equal weights. Add a fixed delay to B. Run once with round robin and once with **least request**, using the same delay and workload so the two runs are comparable.	Least request sends new calls to replicas with fewer active requests. Explain how B's share, active requests, and client p95 differ between the two runs.
Failure and recovery	Round robin, no delay. Configure active health checks and record their interval, timeout, and failure/recovery thresholds. During traffic, stop B, then restart it.	Show redistribution and client errors. Relate how long removal and recovery took to your health-check settings.

A higher weight does not add capacity, and a slow replica can still pass health checks. Explain what you measured; no particular result is required.

Summarise all runs in one table, and check one distribution result against Envoy's access logs for the same time range:

Run / interface	Policy and fault	Target / actual requests per second	A/B/C traffic shares	Client p95	Error rate	Explanation

Conclusions:

1. Which strategy would you choose for HTTP and for gRPC, and why? Use your results.
2. Give one workload where you would choose differently.
3. What happens if Envoy stops? Suggest one improvement; implementation is not required.

Submit

- Code/configuration, fixed-rate test client, exported dashboard, and README with setup, experiment, and recovery commands.
- A short report: architecture and shared state, health-check settings, metric/query details, baselines, results table, two proxy/backend log examples, and conclusions. Aim for 2–3 pages of explanation, excluding charts.

Cite reused work and follow course academic-integrity policy. Exclude dependencies and data volumes.

Optional: try consistent hashing for either interface. Add a fourth replica and show which product IDs change destination.

References: A2, Envoy balancing policies, Envoy health checks.